ENTERPRISE AGENTIC SYSTEMS: FOUNDATIONS

Day 1: Foundations of Agentic Logic

Autonomy, memory, and retrieval: the vocabulary and risk surface everything else this week builds on

DAY 1

Today's agenda

- 09:30 – 09:40 · Welcome & why architecture matters beyond the prompt
- 09:40 – 10:40 · Terminology: a shared vocabulary
- 10:40 – 11:00 · Break
- 11:00 – 12:00 · The Autonomy Spectrum
- 12:00 – 12:45 · Agent Memory Architecture, part 1
- 12:45 – 13:45 · Lunch
- 13:45 – 14:10 · Agent Memory Architecture, part 2
- 14:10 – 15:00 · Retrieval Architecture
- 15:00 – 15:15 · Break
- 15:15 – 16:30 · Workshop, recap & Q&A

CONTEXT

From proof-of-concept to production

- Chat interfaces to LLMs were the first wave. These are useful, but bounded and supervised
- "Agentic" systems act: they plan, call tools, and pursue a goal with less supervision
- A clever prompt is not an architecture. Production needs a framework for memory, retrieval, orchestration, and safety
- **This week bridges probabilistic AI outputs and deterministic enterprise requirements which is the theme to keep returning to**

1

Terminology

A shared vocabulary for the week: you rate yourself, teach each other, play a game and test a few myths

ACTIVITY

Rate yourself

- Take the sheet and work quietly
- Mark each term: never heard, heard, could explain
- Star three terms you want explained
- Tick three you could explain to a colleague

ACTIVITY

Have a wander

- Find someone who ticked your starred term
- Ask for an explanation in plain language
- No words from the sheet allowed in the explanation
- Swap, then find a new partner

DISCUSSION

Pull-back: the slippery words

- Which terms kept coming up as tricky?
- Which explanations worked best?
- Where did two people disagree?
- Agree three shared definitions

TERMINOLOGY

Things to remember

- Grounding makes answers checkable, not correct
- Less randomness is not the same as repeatable
- A prompt is an instruction, not an enforcement point
- A model has no memory between calls
- A standard describes how to connect, not whether to trust
- A bigger window costs more and is still not memory

TERMINOLOGY

Working definitions for the week

- Agent: a model in a loop that chooses its own steps and tools to pursue a goal
- Guardrail: a check on an input, an output or an action; it has an error rate
- Deterministic: the same input always gives the same output
- Drift: behaviour changing without anyone deciding it should
- Handout: the glossary (terminology-glossary.pdf) is the reference all week

2

The Autonomy Spectrum

Assistive → Semi-autonomous → Fully autonomous, and why the boundary matters for risk and controls

THE AUTONOMY SPECTRUM

Three points on a spectrum, not three categories

Mode	What it does	Human role	Typical risk control
Assistive	Suggests; human executes every action	Approves every step	Standard change control
Semi-autonomous	Executes within a scoped plan; flags exceptions	Approves plan or exceptions	Guardrails + escalation triggers
Fully autonomous	Plans and executes end-to-end toward a goal	Sets goal, reviews after the fact	Sandboxing, audit trail, kill switch

THE AUTONOMY SPECTRUM

Reading a system's true autonomy

- Don't take the vendor's label at face value. You need to read what happens on failure
- Assistive: failure = a bad suggestion, ignored by a human
- Semi-autonomous: failure = a bad action inside a scoped blast radius
- **Fully autonomous: failure = a chain of bad actions before anyone notices**
- **Architecture review question: "what is the blast radius if this agent is wrong for one hour, unsupervised?"**

?

QUICK POLL

Think of one AI initiative in your organisation.

Where does it sit on the autonomy spectrum today and where is it drifting?

D

DISCUSSION

Autonomy spectrum: has anyone seen a system drift further than its label suggests?

D

DISCUSSION

Who in your organisation currently owns the decision to move a system from assistive to semi-autonomous? Is that decision visible anywhere?

3

Agent Memory Architecture

Context windows, semantic caching, and compaction and the token-cost trade-off underneath each one

AGENT MEMORY

Context window mechanics

- Everything the model "knows" for a turn lives in the context window. That means nothing persists on its own.
- Longer context is not free memory. Cost and latency scale with tokens in, and quality can degrade well before the limit
- **"Lost in the middle": models attend less reliably to content buried mid-context**
- Architectural implication: memory design is a placement problem. We need to consider what goes in the window, when, and in what order

AGENT MEMORY

Semantic caching & compaction

Semantic caching

- Reuses prior responses for semantically similar queries, not just exact matches
- Cuts cost and latency for common query patterns
- Risk: a stale cached answer served for a "similar enough" question

Compaction strategies

- Summarise or discard older turns as a conversation grows
- Trade-off: shorter context vs losing detail the agent may need later
- Risk: compacting away an earlier constraint or approval the agent should still honour

AGENT MEMORY

Memory tiers: capability vs token cost

Tier	What it holds	Cost profile	Use it for
Working / context window	Current turn's live state	Highest: paid every call	Immediate task reasoning
Session memory	This conversation's history	Medium: grows with compaction	Continuity within one interaction
Long-term / persistent store	Facts, preferences, prior decisions	Low per-call, indexed lookup	Cross-session continuity, personalisation
Retrieved knowledge (RAG)	External documents, on demand	Pay only when retrieved	Facts too large or sensitive to keep resident

AGENT MEMORY

Where each tier lives: managed cloud or on-premises

Tier	Managed cloud (e.g. Bedrock)	Self-hosted (e.g. OpenShift)	Write this requirement either way
Working context	Prompt text goes to the provider's endpoint, in a region you choose	Stays in your estate; you run and patch the model server	Where inference runs and what is retained
Session memory	Provider- or app-managed; check retention defaults	Your cache or database; you set the wipe rules	Retention period and deletion at session end
Long-term store	Managed service; encryption keys may be yours or theirs	Your database; you own backup, access and keys	Who holds the keys and who can read it
Retrieved knowledge	Index copied into the cloud account	Index built beside the source systems	Access controls match the source system

D

DISCUSSION

If context gets compacted mid-conversation, what has to be true for a compliance instruction to survive?

D

DISCUSSION

Token cost tends to be the reason memory architecture gets attention from finance. What's the reason it should get attention from your teams specifically?

4

Retrieval Architecture

From RAG to Agentic RAG, hybrid search, and security trimming for sensitive documents

RETRIEVAL ARCHITECTURE

RAG → Agentic RAG

- Classic RAG: retrieve once, stuff into the prompt, generate. This is a fixed pipeline
- Agentic RAG: the agent decides whether, when, and what to retrieve and can retrieve again if the first pass was insufficient
- **This turns retrieval from a preprocessing step into a tool the agent reasons about**
- Consequence: retrieval decisions now need the same guardrails as any other tool call

RETRIEVAL ARCHITECTURE

Hybrid search

- Keyword (lexical) search: precise, explainable, weak on paraphrase and intent
- Vector (semantic) search: captures meaning and paraphrase, weaker on exact terms, codes, or IDs
- **Hybrid search combines both, typically re-ranked. This is the practical default for enterprise document sets**
- For a bank: policy numbers, account references, and regulation clause numbers are exactly where lexical search still wins

G

GUIDED WALKTHROUGH

Hybrid search, live

A short instructor-led demo, roughly 15 minutes. Run the same query as pure vector search and as hybrid search, and let the room see the exact-match gap.

RETRIEVAL ARCHITECTURE

Security trimming

- The retriever must enforce the same access controls as the source system. Never trust the document index alone
- Trim at query time: a user or agent should never retrieve a chunk they couldn't open directly
- **Failure mode: document-level security is enforced on the file share, but the vector index was built with a service account that could see everything**
- Architecture review question: "who audited what this retriever can see, and does it match who the requesting identity can see?"

D

DISCUSSION

Could you find, today, every document a given RAG pipeline can see?

D

DISCUSSION

Agentic RAG lets the agent decide when to retrieve again. What's the difference between that and an agent deciding to call any other tool a second time?

CASE STUDY

Optional: the mortgage pre-screening assistant

An alternate scenario for the workshop if a group doesn't have its own use case: a drafting aid that quietly became the decision-maker. Full scenario and discussion questions: case-studies.md.

WORKSHOP

Now: apply it to a CSOB use case

- In groups: take one AI use case from your own pipeline (or one from this morning's poll)
- Place it on the autonomy spectrum and say where it's likely to drift
- Sketch its memory tiers - what's in context, session, long-term, and retrieved - and who operates each layer
- Flag one retrieval security-trimming question you'd want answered before this goes to production

Day 1 recap

- The autonomy spectrum is about blast radius on failure, not how impressive the model is
- Memory design is a placement problem across four tiers, each with a different cost profile
- Agentic RAG turns retrieval into a tool call. It needs the same guardrails as any other tool
- Security trimming at the retriever is a control most PoCs skip and most incidents come from
- Tomorrow: architectural patterns - single vs multi-agent, and the protocols that connect them